TASK - 6

(laptop as server using http protocol)

Esp is connected to an ultrasonic sensor which measures distance to an object and that data is sent to the laptop the esp and laptop is connected to the same wifi and the data is sent through http protocol

Wi-Fi and HTTP work together; they don't replace each other. **Wi-Fi** is the physical wireless highway that connects ESP32 to network
TCP/IP Sockets and **HTTP** are different types which uses wifi
HTTP is the standard protocol of the World Wide Web. Instead of a continuous raw data stream (sockets), ESP32 will now act like a web browser sending a quick "POST" request to a laptop every 2 seconds.
HTTP using **Flask** (a lightweight Python web framework) on laptop.

Step 1: Install Flask in VS Code

Because HTTP requires handling web requests, we will use a Python library called Flask to handle the heavy lifting on a laptop.

1. Open **VS Code** and open new terminal
2. Type the command and click Enter to install Flask:
   `pip install flask`

Step 2: Write the HTTP Server Code in VS Code

Create a new file in VS Code named http_server.py and paste this code. This sets up a mini-website on the laptop that listens for data coming to an address called /data.

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# This route listens for incoming HTTP POST requests from the ESP32
@app.route('/data', methods=['POST'])
def receive_data():
    try:
        # Get the JSON data sent by the ESP32
        content = request.get_json()

        if content and 'distance' in content:
            distance = content['distance']
            print(f"HTTP Received - Distance: {distance} cm")

            # Send a successful HTTP 200 response back to the ESP32
            return jsonify({"status": "success", "message": "Data
```

```
received"}), 200
        else:
            return jsonify({"status": "fail", "message": "Invalid data
format"}), 400

    except Exception as e:
        return jsonify({"status": "error", "message": str(e)}), 500

if __name__ == '__main__':
    # host='0.0.0.0' allows any device on your Wi-Fi network to access
this server
    # port=5000 is the standard port for Flask
    print("[*] Starting HTTP Flask Server...")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Step 3: Write the HTTP ESP32 Code in Thonny

We will use MicroPython's built-in urequests library to package the sensor data into a standard HTTP POST request.

Open **Thonny**, create a new file, and paste this code:
Note: change wifi details

```
import machine
import time
import network
import urequests  # Handling HTTP requests in MicroPython
import json

# 1. Hardware Pin Setup
trigger = machine.Pin(5, machine.Pin.OUT)
echo = machine.Pin(18, machine.Pin.IN)

# 2. Network Configurations
WIFI_SSID = "YOUR_WIFI_NAME"          # Replace with your Wi-Fi Name
WIFI_PASSWORD ="YOUR_WIFI_PASSWORD"    # Replace with your Wi-Fi
Password
LAPTOP_IP = "192.168.X.X"             # REPLACE WITH YOUR LAPTOP'S IP
ADDRESS

# The HTTP URL pointing to your laptop's Flask server
SERVER_URL = f"http://{LAPTOP_IP}:5000/data"

def get_distance():
    trigger.value(0)
    time.sleep_us(2)
```

```
    trigger.value(1)
    time.sleep_us(10)
    trigger.value(0)

    while echo.value() == 0:
        signaloff = time.ticks_us()
    while echo.value() == 1:
        signalon = time.ticks_us()

    timepassed = signalon - signaloff
    distance = (timepassed * 0.0343) / 2
    return round(distance, 2)

# Connect to Wi-Fi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(WIFI_SSID, WIFI_PASSWORD)

print("Connecting to Wi-Fi...")
while not wlan.isconnected():
    time.sleep(1)
print("Connected to Wi-Fi! ESP32 IP:", wlan.ifconfig()[0])

# Main Loop
while True:
    try:
        dist = get_distance()
        print("Local reading:", dist, "cm")

        # Prepare data in JSON format
        payload = {"distance": dist}
        headers = {"Content-Type": "application/json"}

        # Send the HTTP POST request
        print("Sending HTTP POST to laptop...")
        response = urequests.post(SERVER_URL,
data=json.dumps(payload), headers=headers)

        # Print response code from your laptop (200 means success!)
        print("Server Response Code:", response.status_code)
        print("Server Message:", response.text)

        # Always close the response to free up memory
        response.close()

    except Exception as e:
        print("Network error occurred:", e)
```

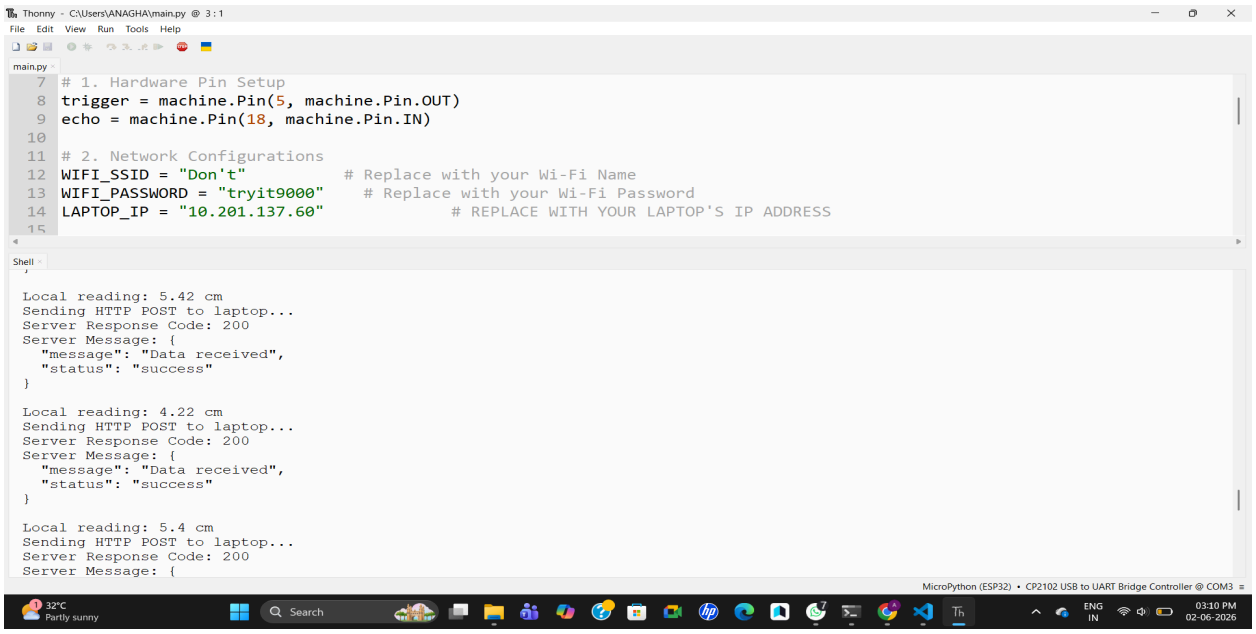
```
time.sleep(2)
```

Step 4: Testing Your New HTTP Setup

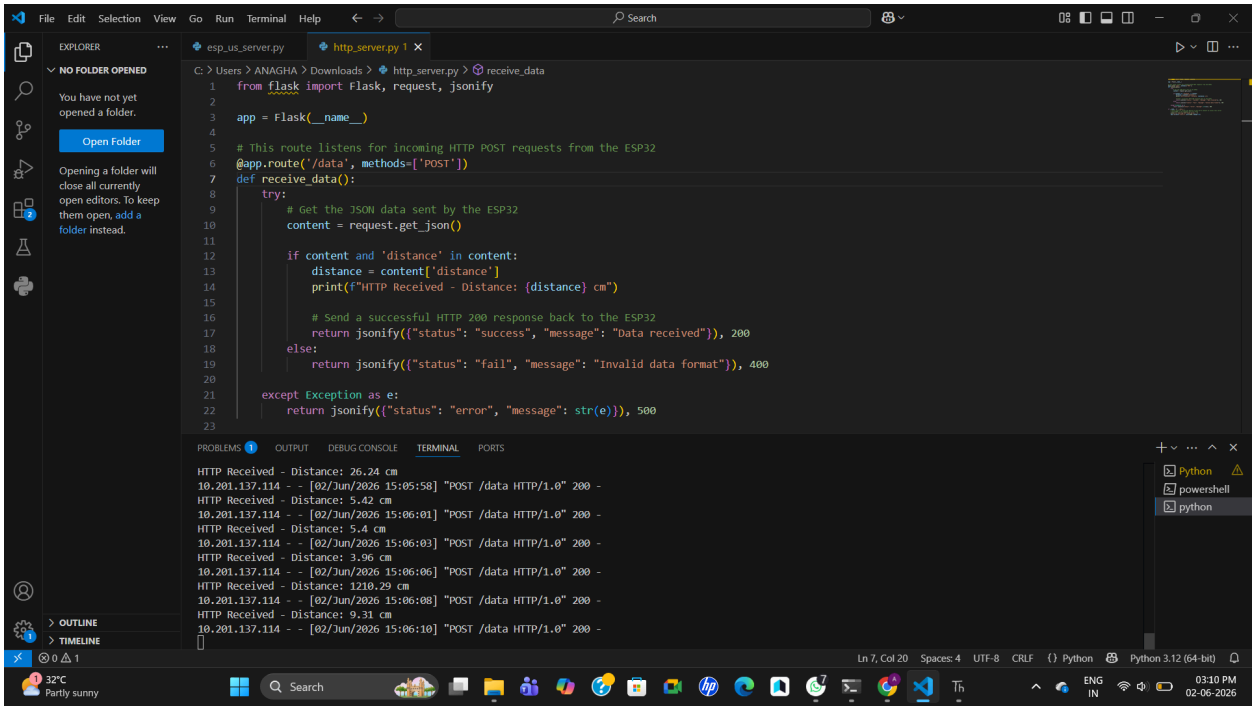
1. **Start the Laptop Server First:** Run http_server.py in VS Code.We will see a message saying * Running on http://127.0.0.1:5000 or http://0.0.0.0:5000. This means our laptop is listening.
2. **Run the ESP32 Code:** Click **Run** in Thonny for MicroPython script.
3. **Observe the Results:** * In Thonny, you should see Server Response Code: 200.
    - In VS Code, our terminal will start actively displaying HTTP Received - Distance: XX cm alongside standard HTTP log entries

OUTPUT

THONNY



VS CODE


File Edit Selection View Go Run Terminal Help
Search
EXPLORER
NO FOLDER OPENED
You have not yet opened a folder.
Open Folder
Opening a folder will close all currently open editors. To keep them open, add a folder instead.
esp_us_server.py
http_server.py 1
C: Users ANAGHA Downloads http_server.py receive_data
1 from flask import Flask, request, jsonify
2
3 app = Flask(__name__)
4
5 # This route listens for incoming HTTP POST requests from the ESP32
6 @app.route('/data', methods=['POST'])
7 def receive_data():
8 try:
9 # Get the JSON data sent by the ESP32
10 content = request.get_json()
11
12 if content and 'distance' in content:
13 distance = content['distance']
14 print(f"HTTP Received - Distance: {distance} cm")
15
16 # Send a successful HTTP 200 response back to the ESP32
17 return jsonify({"status": "success", "message": "Data received"}), 200
18 else:
19 return jsonify({"status": "fail", "message": "Invalid data format"}), 400
20
21 except Exception as e:
22 return jsonify({"status": "error", "message": str(e)}), 500
23
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
HTTP Received - Distance: 26.24 cm
10.201.137.114 - - [02/Jun/2026 15:05:58] "POST /data HTTP/1.0" 200 -
HTTP Received - Distance: 5.42 cm
10.201.137.114 - - [02/Jun/2026 15:06:01] "POST /data HTTP/1.0" 200 -
HTTP Received - Distance: 5.4 cm
10.201.137.114 - - [02/Jun/2026 15:06:03] "POST /data HTTP/1.0" 200 -
HTTP Received - Distance: 3.96 cm
10.201.137.114 - - [02/Jun/2026 15:06:06] "POST /data HTTP/1.0" 200 -
HTTP Received - Distance: 1210.29 cm
10.201.137.114 - - [02/Jun/2026 15:06:08] "POST /data HTTP/1.0" 200 -
HTTP Received - Distance: 9.31 cm
10.201.137.114 - - [02/Jun/2026 15:06:10] "POST /data HTTP/1.0" 200 -
Python
powershell
python
OUTLINE
TIMELINE
Ln 7, Col 20 Spaces: 4 UTF-8 CRLF {} Python Python 3.12 (64-bit)
32°C Partly sunny
Search
ENG IN
03:10 PM 02-06-2026